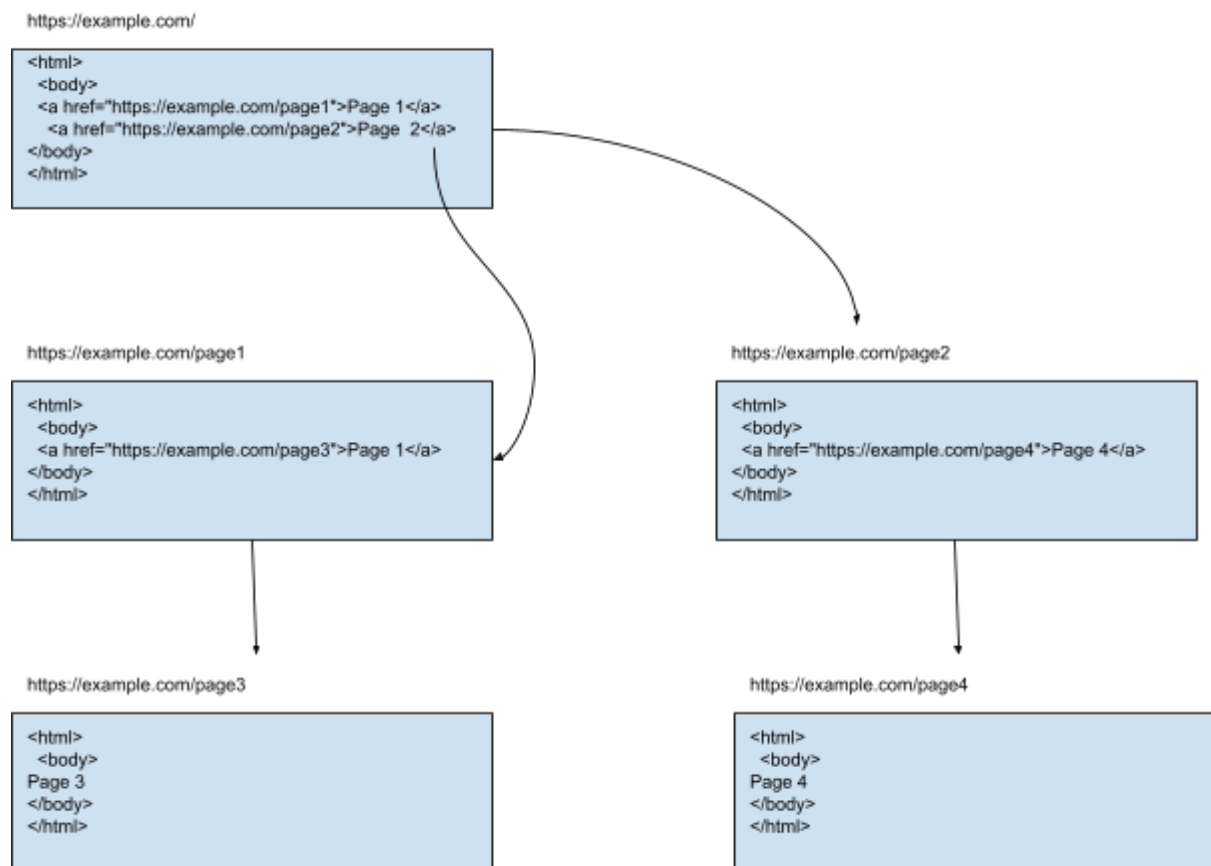


Search Service Spec

El propósito de este trabajo es implementar una solución que permita extraer información de sitios, almacenar la información y permitir la búsqueda de esa información utilizando microservicios.

Existen muchos sitios (por ejemplo los corporativos) que presentan información a los visitantes pero que disponen de un pobre motor de búsqueda. Es decir, no es fácil acceder a los datos.

Por esta razón vamos diseñar e implementar un servicio crawling que visite las páginas de los sitios y las páginas conectadas con tags Anchor (<a/>) para extraer la información. En el siguiente ejemplo se puede ver como desde el *homepage* se puede visitar a las páginas *page1* y *page2*, y desde estas últimas dos se puede acceder a las *page3* y *page4* respectivamente.



¿Qué sitio vamos a inspeccionar?

Se debe poder permitir registrar sitios que nos interese inspeccionar y extraer información. Los parámetros mas importantes son:

- Nombre del sitio
- URL del sitio: Sitio a visitar/crawlear

- Cuantos niveles de hojas vamos a inspeccionar. Por ejemplo, si este parámetro es 2, solo se visitarán la página home, *page1* y *page2*. Si en cambio fuese 3, se incluirían *page3* y *page4*.
- Frecuencia: cada cuánto visitar el sitio.
- Document extractor: Esto es un snippet javascript que deberá extraer datos desde las páginas. Por ejemplo, si utilizamos la librería [Cheerio](#), este sería un potencial snippet:

```
function extract(request, response){
let $=....
return [{
  'name': $('title').text(), // resuelve el tag text en el documento
  HTML
  'url': request.url, // obtiene la url de la pagina visitada
  'description':
    $('meta[property="og:description"]').attr('content') ||
    $('meta[name="description"]').attr('content') || $('p').text()
}]
}
```

- Page resolver (opcional): se espera poder especificar cómo resolver los links a otras páginas que existen dentro de una página. Por ejemplo

```
function pageResolver(request, response){
let $=...
var links = [];
  $("a").each(function ()
  {
    links.push(this.href);
  });
return links
}
```

- Cada cuenta se podrá generar un API Key que autoriza las búsquedas que se discutirá más adelante. La llave puede ser cualquier valor unico aleatorio como un UUID.

¿Cómo se recolecta la información de los sitios?

De acuerdo a los parámetros capturados anteriormente, un job/tarea deberá ser disparada de acuerdo a la frecuencia indicada en la configuración del sitio.

El job visitará las páginas y extraerá su información para ser almacenada en una base de datos.

Por ejemplo, se podrá utilizar cheer.io para extraer la información.

¿Cómo consumir la información?

Se deberá exponer una vista que permita navegar la información capturada del sitio. Es decir, desde la aplicación, se deberá permitir acceder a las diferentes fotos/snapshots que capturo el job/tarea de un sitio. Por cada "foto", se deberá poder navegar por los diferentes documentos extraídos (recordemos que un documento representa los datos de una página visitada).

Por otro lado, se deberá exponer un endpoint que permita buscar información en base a un String (conocido como keyphrase). Esto permitirá integrar este producto en otros sitios.

El endpoint debería recibir un request como el siguiente y retornar una lista de documentos (todos los datos extraídos) que contenga keyphrase enviada:

```
POST /echo/post/json
```

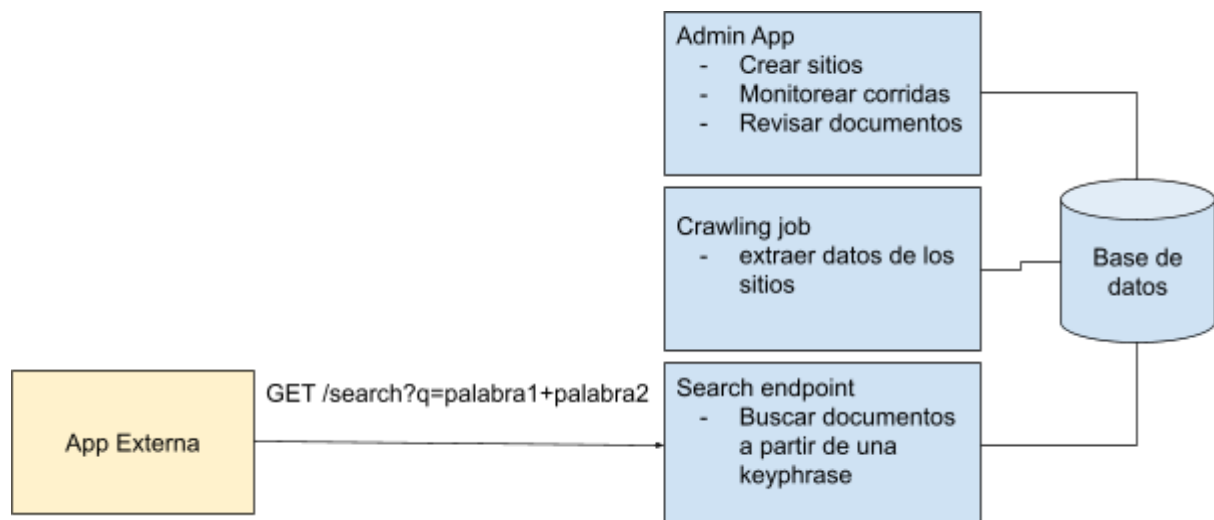
```
Content-Type: application/json
```

```
{ "Id": 78912, "Customer": "Jason Sweet", "Quantity": 1, "Price": 18.00 }
```

```
GET /search?q=palabra1+palabra2
```

```
Authorization: <API Key generada>
```

Un esquema tentativo de la solución es:



Si decide utilizar MongoDB, existe funcionalidad específica ([\\$text](#)) para optimizar las búsquedas basadas en texto.

Cada request de search deberá contar con el API Key generado en el paso 1 para identificar a la cuenta.

¿Como accedemos al sistema?

El sistema deberá permitir accederlo con plataformas de identidad basadas en Single Sign On como [Auth0](#) que permite acceder utilizando Google id.